

Agentic Engineering: Five Papers

A pragmatic reader's guide to five arXiv papers from late May 2026 — skills, tool-use RL, async coordination, lightweight guardrails, and hybrid cloud/edge agents

Content-to-Guide · arXiv digest 2026-05-29

May 2026

Chapter 0: The Through-Line — What Five Papers Agree On

Read five agentic-engineering papers from a single late-May week and a strange thing happens: they stop sounding like five separate results and start sounding like five witnesses describing the same shift. None of them cites the others. They come from different labs, chase different problems — skill files, multimodal RL, async tool use, safety guardrails, edge deployment — and yet they keep arriving at the same handful of convictions. This short chapter names those convictions, because they're more useful as a set than any one paper is alone. The five deep dives follow; this is the map.

The unit of work is the trajectory, not the answer

The oldest habit in LLM evaluation is to judge the final string. Every paper here breaks that habit.

SkillGrad refuses to grade a skill file on output alone and instead reads the entire execution trace to figure out what behavior caused the outcome. **AgentDoG 1.5** makes the same move for safety: a chatbot’s harm lives in its last message, but an agent’s harm lives three tool calls back, so the guardrail audits the whole path. **AsyncTool** scores at the step, sub-task, and task level because a single end-state verdict hides whether the agent was coordinating or thrashing. **AXPO** intervenes inside the rollout, freezing the part of the trajectory that was fine and resampling only the part that broke.

The practical lesson is portable and immediate: if your agent observability still logs prompts and final responses, you are watching the wrong layer. The signal — for improvement, for safety, for debugging — is in the sequence of decisions, and you have to capture it to use it.

Targeted supervision beats brute scale, every time

The most consistent empirical result across the bundle is that spending *less, but in the right place* wins. SkillGrad’s roughly six-dollar refinement loop beats training-based skill evolution. AXPO’s 25% extra resampling budget beats *doubling* the rollouts under the standard recipe, and its 8B model surpasses a 32B base. AgentDoG’s 4B judge, trained on about a thousand samples, rivals frontier closed guardrails and beats a 397B open model. The cloud-device study finds that buying *more* cloud supervision actively degrades accuracy past a surprisingly early point — the frontier model works best as a seasoning, not the main course.

This is the same finding wearing five costumes. The skill being optimized — refining prose, recovering a tool call, recognizing risk, supervising an executor — is narrow and learnable, and general capability bought at scale stumbles into it expensively or not at all. For anyone with a budget, the takeaway is to stop reaching reflexively for the bigger model and start asking where the actual learning signal is, then aim a small, cheap, focused mechanism at exactly that spot.

The failures that matter are agentic, not single-shot

A model that aces single-turn benchmarks can still fall apart the moment it has to loop, wait, or hold state — and each paper documents a failure mode that only exists *because* the thing is an agent. AXPO names the **thinking-acting gap**: models reason fluently but barely attempt tool calls, and fumble them when they do. AsyncTool exposes agents that can’t keep their place while several tool calls are in flight, hallucinating results from impatience. AgentDoG catches risk that accumulates mid-trajectory while the final reply looks clean. The cloud-device work watches a trigger-happy verifier restart its executor into self-sabotage.

None of these show up if you test one prompt, one call, one turn. They're properties of the loop. So the engineering implication is that your evaluation has to have a loop in it — multiple tasks, real latency, long horizons, irreversible actions — or it simply won't surface the failures that will actually bite you in production.

Leverage lives in the scaffolding

Notice what none of these papers do: retrain a foundation model from scratch. The wins come from the layer wrapped *around* the model. SkillGrad optimizes the skill package — Markdown files, not weights. AsyncTool's lesson is to build an orchestration layer that maintains per-task state the model won't hold on its own. AgentDoG bolts a diagnostic guard onto the one checkpoint every framework already exposes. The cloud-device paper's entire result is about *who plans and who executes* and *how often you interrupt* — pure architecture, fixed models.

That's the most empowering thread for a practitioner who can't train a frontier model and never will. The highest-leverage edits are in the harness: the skill files, the state tracking, the guardrail at the boundary, the division of labor between a big advisor and a small worker. Each paper, in its own domain, found more gain in the scaffolding than scale was offering.

What to carry into the five chapters

Read the rest of this guide with three questions in hand. *Where is my agent's trajectory, and am I capturing it? Where is the actual signal, and am I spending my compute and data there instead of everywhere? What's in my scaffolding that a small, cheap, targeted mechanism could do better than a bigger model?* Every chapter ahead is, in some sense, one team's worked answer to those questions.

We start with the most domestic of them — the skill file you might already maintain, and why the one your model generated could be making it dumber.

Chapter 1: SkillGrad — Optimizing Agent Skills Like Gradient Descent

Paper: [SkillGrad: Optimizing Agent Skills Like Gradient Descent](#) · arXiv 2605.27760 · **Code:** github.com/wwwwhy725/SkillGrad

A generated skill file is supposed to make your agent smarter. Often it does the opposite. In one of this paper’s cleaner embarrassments, an LLM was asked to write a spreadsheet skill, then handed that skill back to itself. Accuracy *dropped* — from 62.5% with no skill at all to 55.8% with the “helpful” one. The same backbone, the same tasks, made worse by its own advice. If you maintain a `SKILL.md` somewhere, that result should sting a little.

The deeper point is that a skill file is not a static thing you generate once and trust. It is an artifact you can be wrong about, and therefore an artifact you can *improve* — systematically, not by vibes. That is the whole pitch of **SkillGrad**: take the discipline of gradient descent, the workhorse of model training, and borrow its *shape* to refine the text inside a skill package.

The analogy that does the work

There is no real derivative here. A skill is a folder of natural-language Markdown, not a weight vector, and you cannot take its gradient. What SkillGrad borrows is the *control flow* of optimization, and it maps each piece onto something concrete.

The **parameter** being optimized is the skill package itself — specifically its three layers: the YAML metadata that decides when the skill activates (L1), the always-loaded `SKILL.md` body (L2), and the on-demand `references/*.md` files the agent reads only when relevant (L3). The **loss** is brutally simple: did the agent’s output pass evaluation, yes or no. But a bare pass/fail signal is too sparse to edit prose with, so SkillGrad gathers richer **loss evidence** — the full execution trajectory, not just the verdict.

That evidence becomes a **text gradient**: a diagnoser agent reads a trajectory and writes, in plain language, what behavior caused the outcome and what should change. Run that over a mini-batch of tasks and you get a batch of diagnoses. A **momentum** mechanism carries recurring diagnoses across iterations so the optimizer reinforces patterns that keep showing up instead of chasing one-off noise. Finally a patcher applies the update — a **layer-aware patch** that decides not just *what* to add but *where* it belongs in the L2/L3 hierarchy. Then the loop runs again on the freshly patched skill, which changes future trajectories, which changes the next gradients. Closed loop, just like training.

The two ideas worth stealing

Most prior skill-evolution tools learn only from failures: something broke, diagnose it, patch it. SkillGrad's first real insight is that **successes carry signal too**. The trick is contrast. It deliberately seeds the training set with tasks the *initial* skill failed. So when the current skill succeeds on one, you can diff the two trajectories and ask: what changed? That delta — a verification step that got added, a more robust formula choice, an inspection the agent learned to do first — is a behavior worth *preserving*, not just a problem worth fixing. Without it, an optimizer only ever knows how to apologize; it never learns to protect what's working. Ablating contrastive diagnosis cost about 4 points of held-out accuracy.

The second idea is that **placement is part of the edit**. Because `SKILL.md` is always loaded, dumping every lesson there bloats the context on every future task and dilutes the broadly useful guidance. Narrow, task-specific procedures belong in L3, loaded only when a pointer in L2 says they're relevant. SkillGrad treats this routing as the parameter update itself. The training dynamics are telling: L2 bootstraps fast — roughly 40 lines to 120 in the first iteration — then *plateaus* around 154 lines while L3 keeps growing toward 13k characters. The optimizer stops appending to the always-loaded file and starts filing new material where it won't get in the way. Strip out momentum and that discipline degrades: more text piles into L2, less gets consolidated into L3, and accuracy falls about 6.7 points.

Does it actually work

On SpreadsheetBench Verified and the out-of-domain WikiTableQuestions, SkillGrad beats both training-free settings and the stronger skill-evolution baselines, by an average of 6.7 points. With a weak backbone (gpt-4.1) it jumped from a 37% baseline tie to 54% — the framework rescued a model that the raw skills were actively hurting. It improved skills from two different sources, including one downloaded from a public repo, so the gains aren't an artifact of one lucky starting file. And the optimized spreadsheet skill still won on WikiTQ, suggesting it learned transferable procedure rather than memorizing the test distribution.

Two honest caveats the authors surface. Accuracy peaks around iteration 10 and then *declines* — text updates aren't monotonic, and over-editing starts trading away earlier general rules for narrow ones. And the whole thing is empirical; there's no convergence guarantee, just a loop that behaves like one. The cost, for what it's worth, is real but modest: about USD 6.40 for a full ten-iteration run, no fine-tuning, no validation sweep over candidate skills.

What a skill maintainer can take home

You don't need the multi-agent harness to use the lessons. If you maintain agent skills, three moves are borrowable at any scale.

First, **stop trusting generated skills on faith** — measure them against the no-skill baseline. A skill that loses to no skill is worse than nothing, and you won't know without an eval set of failing tasks.

Second, **mine your successes, not just your failures**. When a skill revision fixes a task that used to break, look at *what behavior* did it — an added verification, an inspection-before-acting habit — and write that into the skill explicitly so it survives the next refactor.

Third, **be deliberate about the L2/L3 boundary**. Keep the always-loaded body short and broadly useful; push narrow procedures and edge cases into reference files gated behind a clear “read this when...” pointer. SkillGrad found that good skills converge on exactly this shape on their own — a small, stable core plus a growing handbook consulted on demand. The executor prompt even enforces it, telling the agent it *must* read a reference before coding when `SKILL.md` points to one, because “past failures show that skipping references leads to silent errors.” That is a maintenance principle you can adopt today, no gradient required.

If SkillGrad is about making the agent's *written knowledge* reliable, the next chapter asks a more basic question: what happens when a multimodal agent barely tries to act at all? AXPO takes on the “thinking–acting gap” — agents that reason fluently but rarely attempt a tool call, and get it wrong when they do.

Chapter 2: AXPO — Closing the Thinking-Acting Gap

Paper: [Agent Explorative Policy Optimization for Multimodal Agentic Reasoning](#) · arXiv 2605.28774 ·

Project page: byungkwanlee.github.io/AXPO-page

A reasoning model that can call tools is supposed to behave like a resourceful problem-solver: think for a while, and when thinking alone won't cut it, reach for a Python interpreter, a web search, or a zoom-in on the image. But watch one of these models actually train under a standard reinforcement-learning recipe and a strange pattern emerges. The model barely reaches for its tools at all, and on the rare occasions it does, it tends to fumble them all at once. The very behavior that makes an agent an agent is the behavior the training process quietly starves.

This is the problem AXPO sets out to name and fix. It calls the underlying asymmetry the **Thinking-Acting Gap**, and its contribution is a surgical change to how rollouts are sampled during RL that closes it.

Two symptoms hiding in plain sight

The gap comes from a structural difference between the model's two modes. Thinking is the safe default: every token lives inside the model's own space, and a bad chain of thought can be corrected by trying a different one. A tool call is something else entirely. A short, fragile token sequence triggers an external response the model can't predict, and a tiny change — a wrong Python indent, a slightly-off search query — produces a wildly different outcome. Reasoning models are heavily post-trained to think and barely post-trained to act.

When you run the dominant RL recipe, **GRPO** (Group Relative Policy Optimization), on top of this asymmetry, two measurable symptoms show up and stay put across the whole training run:

- **Tool use is under-attempted.** Across eight rollouts per question, only about 20-35% contain a tool call. GRPO applies no pressure to grow that minority, so the defining agentic behavior stays rare.
- **When tools are attempted, the whole group tends to fail together.** Conditional on a question where some rollout reached for a tool, the tool-using rollouts are *all-wrong* roughly 40% of the time, versus about 25% for the no-tool rollouts.

That second symptom is the killer, and it's worth understanding why. GRPO computes each rollout's advantage by normalizing its reward against the group mean. If every tool-using rollout in the group fails, the advantage on those tool-call tokens collapses to zero — no learning signal. If some no-tool rollouts succeed while the tool-using ones fail, the tool-call tokens get *negative* advantage, actively

teaching the model to stop reaching for tools. Either way, the gradient goes silent on exactly the tokens that needed it most: the questions where a tool would have helped.

The fix: resample the action, not the whole trajectory

The insight that unlocks AXPO is almost embarrassingly simple. When a tool-using rollout fails, the *thinking* that led up to the call is often perfectly sound — the failure was in the call itself. So instead of throwing away the whole trajectory and re-deriving it from scratch, AXPO freezes the thinking prefix right up to the opening `<tool_call>` tag and resamples only the tool call and everything after it. Fixing a prefix and drawing 16 continuations yields 2.9 to 3.4 distinct semantic clusters of tool calls, confirming this is a real divergence point: the thinking doesn't lock in the action, so the exploration just needs to be supplied.

AXPO supplies it with three targeted choices, all bolted onto a small extra rollout budget:

- **Where to resample:** only the all-wrong tool-using subgroups, where a single recovered success flips a dead-zero advantage into a positive one. Groups that already have a correct tool-using rollout are left alone — they'd gain little.
- **Which prefix:** the most uncertain ones, ranked by the model's mean confidence over the tool-call tokens. Low-confidence calls are the ones most worth re-rolling.
- **How the resamples enter the gradient:** continuations get a normal per-prefix GRPO advantage, while the shared source prefix gets a binary *recovery* reward — it fires if any resampled continuation succeeded. That split avoids a gradient conflict where the failed original would penalize the same prefix a successful resample rewards.

Because every resampled continuation is tool-using *by construction*, the fraction of budget wasted on non-tool rollouts vanishes — and the paper proves that at any fixed budget, resampling from a committed prefix dominates raw from-scratch sampling at recovering a correct tool-using rollout.

What it buys you

Across nine multimodal benchmarks and three sizes of Qwen3-VL-Thinking, SFT + AXPO beats the standard SFT + GRPO recipe at every scale — Pass@4 gains of +2.8, +2.3, and +1.8 points at 2B, 4B, and 8B. The headline result is a parameter-efficiency one: an 8B model trained with AXPO reaches 99% of the *32B base model's* Pass@1 and actually surpasses it on Pass@4, with 4x fewer parameters. The 2B model, similarly, matches the 8B base.

The gains concentrate on perception tasks, where an image-zoom tool most directly decides correctness, and stay modest on pure reasoning, where the bottleneck is the thinking. During training, both symptoms reverse: tool-use rate climbs 28 points and the all-wrong rate sits about 17 points below GRPO, because resampling flips roughly 12% of dead subgroups per step back into ones with a correct continuation.

The most pointed control: 25% extra resampling budget with AXPO beat *doubling* the rollout budget under GRPO. The win is about where compute is spent, not how much. Reward-shaping alternatives — adding tool bonuses or penalties — all fell below the GRPO baseline, suggesting you can’t bribe a model across this gap with reward terms.

What a practitioner can borrow

Even if you’re never going to train a multimodal agent, the diagnostics here are the takeaway. If you run agentic RL and your tool-use rate is flat in the 20-35% band while your eval numbers stall, you are very likely staring at this gap. The cheap check is the *all-wrong rate on tool-using subgroups* — measure it conditional on tool attempts, and if it’s running near 40% you have entire groups producing zero gradient on the tokens you care about.

The fix generalizes beyond vision: anywhere a committed “intent” boundary precedes a high-variance action — an API call, a code block, a function invocation — you can freeze the prefix and resample only the action, target the all-wrong subgroups, rank by token-level confidence, and split the prefix credit from the continuation credit to dodge the sign conflict. It’s a rollout-distribution change, not a reward hack, which is precisely why it sticks.

AXPO is about making a single agent *decide* to reach for a tool and reach well. The next chapter turns to a different agentic blind spot: what an agent does while it *waits* — Chapter 3 introduces AsyncTool, a benchmark for whether agents put idle time to work across concurrent tasks instead of stalling on slow, asynchronous tool calls.

Chapter 3: AsyncTool — Can Agents Multitask While Tools Run?

Paper: [AsyncTool: Evaluating the Asynchronous Function Calling Capability under Multi-Task Scenarios](#) · arXiv 2605.27995 · **Code:** github.com/StoKou/repo-async-tool

Picture an agent handed two errands at once. Add a stock to a watchlist and fund the account; separately, find a support ticket and bump its priority. Each step calls a tool, and each tool takes a beat to respond. A smart human assistant fires off the first call, and while that result is still in flight, starts on the second errand. A blocking, one-thing-at-a-time assistant just sits there, staring at the spinner, until the first call returns. Same tools, same tasks, but one of them is waiting on dead air for half the job.

Almost every tool-use benchmark we evaluate agents against assumes the second kind of assistant — and assumes that dead air doesn't exist. Tools respond instantly, tasks arrive one at a time, and the only question is whether the agent picked the right function with the right arguments. **AsyncTool** is the first benchmark built to break that assumption. It hands the agent several heterogeneous tasks simultaneously, makes every tool call return its result *after a delay*, and then asks the question nobody else was measuring: when a tool is busy, does the agent use that idle time to advance something else, or does it stall?

What the benchmark actually does

The setup is deliberately realistic. Each task is a sequence of tool calls with internal ordering constraints — call `get_symbol_by_name` before `add_to_watchlist`, because the second call needs the symbol the first one returns. Multiple such tasks are presented at once, and they're independent of each other, so an agent is free to interleave them. The twist is that when the agent issues a call, the environment doesn't hand back a result. It says, in effect, “not ready yet.” The agent must then decide: keep waiting on this task, or switch to a different one and make progress there. When the earlier result finally lands, a capable agent picks the original task back up at the right spot. This is what the authors call **asynchronous tool calling** — coordinating several pending tasks under delayed, sometimes out-of-order, tool feedback.

Building the dataset took some care. The team started from two existing tool-use benchmarks (NESTFUL and BFCLv3), pulled out 12 tools and 358 single-task trajectories, cleaned them up with Gemini 2.5 Pro plus rounds of human annotation, then composed them into 712 multi-task instances — pairs and triples of tasks, mixing similar and cross-domain combinations. Crucially, the *same* tasks can be run in a normal instant-response mode or in the delayed async mode, so degradation from latency can be isolated cleanly.

Scoring happens at three altitudes. **Step level** checks raw tool-call correctness — did you name the right function with the right parameters (reported as F1 scores). Sub-task level checks whether each individual errand actually completed, both in its call trajectory and in the resulting environment state. Task level is the strict one: a task counts only if *every* sub-task inside it succeeds. On top of accuracy, they add an efficiency lens — a “same-task streak” metric that counts how many turns in a row a model stays glued to one task before switching. Lower means more interleaving.

So what — the result that should worry you

Across 19 models, async execution hammers performance. The strongest model overall, GPT-4.1, lands at just 38 out of 100 on the full task-level score. Gemini 2.5 Pro sits around 32, GPT-4o and GPT-5 near 31. The open-source field falls off a cliff: most sub-20B models score in single digits, and even a 70B Llama scores under 3 because it can complete individual tool calls fine but can’t hold multiple tasks in its head. The large open model DeepSeek-V3.1-Terminus is the standout exception, roughly matching the closed-source pack at 29.

The instructive part is the failure-mode breakdown, because it maps directly onto how production agents misbehave. Weaker models lack *temporal awareness*: they fire the next call in a task immediately, before the previous result has come back, then fabricate parameters based on what they *assume* the missing result said — straight-up hallucination born of impatience. Others execute the most recently mentioned task first and quietly forget the earlier one, a neglect problem that gets worse with three tasks than two. A third pattern is tool confusion across tasks — invoking a flight-booking tool mid-way through a data-processing task — which cascades because the model rarely self-corrects once it’s lost the thread.

And here’s the finding that reframes the whole problem: switching often is *not* the goal. Some small models have low same-task streaks — they interleave constantly — yet still score poorly, because thrashing between tasks wrecks their ability to track which task is which. Meanwhile DeepSeek interleaves less but scores well. Effective async behavior isn’t “switch a lot.” It’s switch *at the right moment* while preserving state. Accuracy and scheduling efficiency are two separate axes, and you need both.

What you can try with this

If you’re building multi-task agent systems, the practical takeaways land hard.

First, **stop assuming your tool calls are free and instant**. If your orchestration blocks on every call — issue, await, issue, await — you’re paying full latency on a critical path that an async-capable controller could overlap. The paper’s whole premise is that idle wait time is recoverable throughput you’re leaving on the table.

Second, **measure the temporal dimension in your own evals**, not just call correctness. A model can ace single-call accuracy and still collapse the moment two tasks compete. Borrow AsyncTool’s framing: run the same task suite in instant-response mode and in delayed mode, and watch the gap. That gap *is* your async coordination debt. Track a same-task-streak-style metric alongside accuracy so you can see whether failures come from bad tool calls or bad scheduling — they need different fixes.

Third, **design the orchestration layer to do what weak models won’t do for themselves**: maintain explicit per-task state, enforce intra-task dependency ordering so a call can’t consume a result that hasn’t arrived, and gate task resumption on the actual arrival of pending results rather than on the model’s optimism. The models that win here behave like coordinators tracking several open threads; if your scaffolding makes that state explicit instead of trusting the model to hold it implicitly, you get much of the benefit even from a mid-tier model. Note too that latency itself compounds difficulty — longer delays mean more interaction turns and more chances to lose the thread — so the orchestration discipline matters more, not less, as your real tools get slower.

The blunt verdict: today’s agents are good at calling tools and bad at waiting on them. Once you accept that real tools have latency and real users send more than one request at a time, “can it call the function” stops being the interesting question. “Can it keep its place while three things are in flight” is the one that decides whether your system feels fast or feels stuck.

Coordination under pressure is one kind of agent failure. The next chapter turns to a different one — keeping agents safe and on-policy — where AgentDoG 1.5 shows that a lightweight guardrail model trained on roughly a thousand samples can rival closed-source safety systems.

Chapter 4: AgentDoG 1.5 — Lightweight Guardrails That Diagnose

Paper: [AgentDoG 1.5: A Lightweight and Scalable Alignment Framework for AI Agent Safety and Security](#) · arXiv 2605.29801 · **Code:** github.com/Al45Lab/AgentDoG

A modern agent does not fail the way a chatbot fails. A chatbot says something it shouldn't and the harm stops there. An agent, by contrast, accumulates risk across a live trajectory: it reads a tool description, calls a tool, observes a result, reasons about it, calls another tool, and somewhere in that chain it decides to run `sudo rm -rf /` or quietly leaks a credential file. The final reply might look perfectly benign. The damage already happened three tool calls ago.

That gap — between where a risk enters and where it surfaces — is the problem AgentDoG 1.5 is built to close. It is a family of small judge models (0.8B, 2B, 4B, and 8B parameters) from Shanghai AI Lab that audit an agent's *entire execution trace* and decide whether anything along the way was unsafe. The headline result is almost suspicious: trained on roughly **1,000 samples**, the 4B variant matches or beats GPT-5.x-class closed guardrails on agentic safety benchmarks, and the 0.8B model — small enough to run as a sidecar — already outperforms much larger general-purpose models at the diagnostic task. The whole thing is openly released, models and datasets both.

Why a tiny guardrail is the interesting part

The instinct in safety work is to reach for a bigger model. AgentDoG 1.5 argues the opposite: for trajectory-level judgment, *targeted supervision beats scale*. A 4B model fine-tuned on the right thousand examples surpasses a 397B open-source model and rivals frontier closed models, because the skill being learned — recognizing risky agent behavior in context — is narrow and learnable, not something general capability stumbles into for free.

For anyone deploying agents in production, that changes the economics of safety. A frontier guardrail means an API call (cost, latency, a third party in your trust boundary) on every checkpoint. A 0.8B–4B judge you host yourself can sit inline. The paper's own RL-environment work makes the scale concrete: a standard 8-core machine ran over 10,000 concurrent simulated environments with peak memory under 2.5 GB — roughly two orders of magnitude cheaper than Docker-level setups like SWE-Bench. Cheap enough that “audit every agent run” stops being a budget conversation.

Provenance, not a binary verdict

The second idea worth borrowing is that **AgentDoG doesn't just say safe or unsafe — it says why**. Most guard models emit a single label. AgentDoG decomposes every unsafe trajectory along three axes drawn from a structured taxonomy of agentic risk:

- **Risk source** — *where* the risk entered: a malicious user instruction, an indirect prompt injection in a tool description, session-state contamination, a poisoned dependency or MCP server, or the agent's own flawed reasoning.
- **Failure mode** — *how* the agent failed: an over-privileged action, missing validation of a tool's output, an unsafe shell execution, approval bypass, cross-tool attack chaining.
- **Real-world harm** — *what* it could cost: privacy leakage, system-integrity damage, financial loss, compliance or auditability violations.

That separation matters operationally. A flat “unsafe” label tells your on-call engineer nothing. “Indirect prompt injection in a tool response led to an over-privileged file deletion with system-integrity harm” tells them exactly where to look and what to patch. The taxonomy is also deliberately extensible: the three top-level dimensions stay fixed while leaf categories get customized per execution setting — which is how the same framework stretched to cover both stateful assistant agents (the ATBench-Claw benchmark) and coding agents acting on repositories and shells (ATBench-Codex) without redefining the task.

Interestingly, teaching the model to diagnose made it a *better binary classifier* too. The unified variant that learned the fine-grained labels posted the team's best safe/unsafe accuracy — the reasoning scaffolding turned out to be useful intermediate structure for the simpler decision.

Where it bolts onto your stack

The most directly usable contribution is the deployment pattern: AgentDoG 1.5 as a **training-free online guardrail** at the *Pre-Reply* checkpoint — the moment just before the agent hands its final answer back to the user.

The choice is pragmatic, and the reasoning is worth stealing. Checking after every tool call would pile up latency across the dozens-to-hundreds of serial calls a long task makes, and the developer hooks for mid-loop interception vary wildly across frameworks. Pre-Reply, by contrast, exists in essentially every agent framework, so one guardrail integrates almost everywhere with minimal glue. A proxy mirrors the runtime event stream — tool calls, observations, intermediate reasoning, the evolving draft — into a trajectory representation, AgentDoG judges it, and safe replies pass while unsafe ones are blocked. A dashboard logs every decision so operators can trace a block back to the events that caused it.

The numbers are honest about the trade-off. On the ClawSafety benchmark the 4B guardrail cut the residual unsafe-delivery rate from 56% to 19%; on a CIK personal-agent attack set, from 94% to 43%. It generates longer outputs than a bare label-classifier, but time-to-first-token stays sub-second, and because the cost is paid *once* at delivery rather than per tool call, it's affordable in practice. Note the ceiling, too: a Pre-Reply guard can only stop harms that are still preventable at final reply. Damage already committed to disk or sent over the wire mid-trajectory is out of its reach — which is an argument for pairing it with cheaper local rule-rails, not replacing them.

What to try

If you run agents, the practical moves are clear. First, treat the *trajectory* as the unit of safety review, not the prompt and not the final string — wire a proxy that captures the event stream and feed it to a judge before delivery. Second, demand provenance from whatever guardrail you adopt: a three-axis “source / failure / harm” verdict turns alerts into actionable triage. Third, take the small-model economics seriously — a self-hosted 0.8B–4B judge inline is a different cost and trust profile than a frontier API call per turn, and the benchmarks say you give up little. And if you train your own agents, AgentDoG’s filtering recipe — use the judge to keep only the safety examples whose “safe” trajectory genuinely neutralizes the injected risk — cut harmful behavior far more than simply piling on more synthetic safety data, while preserving function-calling skill.

The throughline of this bundle so far has been agents that do more, faster, in more places. AgentDoG is the counterweight: the cheaper and more autonomous agents get, the more you need a cheap, fast, diagnostic layer watching them. The next chapter pushes the deployment question one step further — what happens when agents stop living in one place at all, and split their work across the cloud and the device in your pocket.

Chapter 5: When Cloud Agents Meet Device Agents

Paper: [When Cloud Agents Meet Device Agents: Lessons from Hybrid Multi-Agent Systems](#) · arXiv 2605.30102

Every time you point an agent at a long, messy task, you are quietly buying tokens. A frontier model in the cloud is brilliant and forgetful in equal measure, and agentic work is the worst case for its pricing: long horizons, iterative loops, tool call after tool call, each one re-reading a growing transcript at per-token rates. The bill climbs even when the work is dull. Meanwhile a small model sits on your laptop or phone, nearly free to run, willing to grind through repetitive steps, but liable to wander off a cliff on anything that needs real judgment. The obvious move is to use both. The non-obvious part is how.

This study takes that “use both” instinct and makes it rigorous. The authors build **hybrid multi-agent systems** that split work across a cloud LLM and an on-device **small language model (SLM)**, then map how each design choice slides the system along a three-way Pareto frontier: task accuracy, dollar cost, and the energy your edge device burns. The headline finding is bracing for anyone hoping for a recipe: there is no single best architecture. The right one depends on the task.

Two ways to split the work

The paper studies two architectures, both built on a simple division of labor. The token-hungry part of agentic work, the multi-turn reason-act-observe loop where the agent actually touches tools and the environment, runs on the cheap on-device model. The expensive cloud model is invited in only intermittently, as a supervisor. A single knob, the verification interval, controls how often the cloud is consulted, and therefore how much you spend. Crucially, every time the supervisor steps in, it resets the executor’s context, which keeps the device’s memory footprint bounded.

The two differ in what the supervisor does. In **PEVR** (Plan-Execute-Verify-Replan), the cloud model writes a detailed, step-by-step plan up front, hands it to the device executor, and periodically checks whether execution still matches the plan. When it drifts, the cloud issues a fresh replan. This is classic centralized orchestration: the cloud owns the strategy. In **EVA** (Execute-Verify-Advise), there is no initial plan. The device model just starts working from the raw query, and the cloud occasionally summarizes what’s happened and offers advisory feedback on how to proceed, then lets the device continue from that summary. PEVR commands; EVA coaches.

Why the same trick wins and loses

Tested across multi-hop QA (HotpotQA), fan-out aggregation (FanOutQA), and a stateful app-automation benchmark (AppWorld), the two architectures cleanly trade places. PEVR dominates on the interactive UI task; EVA dominates on the deep-search tasks. The mechanism behind the split is the interesting part.

In a stateful world like AppWorld, an early wrong move can be irreversible, like emailing the wrong person. Here PEVR's detailed up-front plan with explicit tool calls and control flow is worth a lot. It performs better even with zero interventions, because the plan itself keeps the device executor on rails. Aggressive supervision pays off because adherence to a good plan is a meaningful signal that execution is on track.

Deep search is the opposite. Reasoning is fragile and easily disrupted, and PEVR's verifier turns out to be trigger-happy, restarting the executor repeatedly within a single task. Restart count correlates negatively with accuracy, so PEVR's eagerness becomes self-sabotage, often dropping below even the device-only baseline. EVA's lighter, more conservative touch wins. A telling ablation: strip the summarization out of EVA and performance barely moves, which means the gap isn't about how context is compressed. It's about the verification policy itself. The plan-based verifier has a higher false-positive rate (it intervenes when it shouldn't); the query-based one is more cautious. On fragile long-horizon reasoning, caution is a feature.

And more cloud is not more better. Both architectures show an optimal verification interval beyond which buying more supervision actively degrades results. The cloud's compute is a seasoning, not a main course.

What an engineer actually gets to control

If you architect agent systems under cost, latency, or privacy pressure, the practical levers here are concrete and worth borrowing:

- **Push the loop to the edge, keep supervision in the cloud.** The paper tests the symmetric alternative, cloud executor plus on-device supervisor, and finds it worse on both accuracy and cost. Execution is where the tokens are; that's exactly what you want off the expensive meter. The asymmetry isn't arbitrary.
- **Tune the verification interval as your cost dial.** One parameter moves you along the frontier. Verify aggressively on stateful tasks that tolerate restarts; verify sparingly on long reasoning chains that don't.
- **Match the intervention style to the task.** Binding plans for irreversible, stateful environments. Advisory summaries for exploratory search.
- **Lean on context resets for memory, not just accuracy.** Because supervision resets the executor's context, the hybrid systems hold KV-cache growth nearly flat where monolithic device

agents balloon and eventually hit out-of-context failures (roughly 20% of runs at 80 turns). The paper estimates footprints that map cleanly onto real device budgets, mobile in the single-digit gigabytes, laptops up to the mid-teens. Resets keep both architectures inside those budgets while cutting edge energy by up to 3x versus a device-only baseline. The cloud LLM here works like a small frontier “advisor” the way Anthropic’s Opus-boots-Sonnet pattern does, except the junior model is running on your phone.

What’s most useful is a negative result dressed as a Venn diagram: the hybrid system solves tasks that neither the pure-cloud nor the pure-device agent can solve alone. A hybrid is genuinely different from a router that merely picks one model per query. But no fixed setup wins everywhere; the union of all configurations beats any single one. The honest takeaway is that the next gain comes from a system that picks its own topology per task, dynamically, rather than from any one clever static design.

So the thing to try is not to adopt PEVR or EVA wholesale. It’s to instrument your own agent the way this paper does: treat verification frequency as a tunable, measure accuracy against both dollars and joules, and watch where over-supervision starts costing you. The frontier model’s best move is often to stay quiet and let the small one work.

This is the last of the five papers, and it lands on a useful note of humility. The flashy frontier model is not always the answer, and a bigger cloud budget does not buy a better agent past a surprisingly early point. The leverage is in the architecture, in who plans and who executes, in how often you interrupt, in what you throw away between turns. A short synthesis chapter sits ahead of these five to tie the threads together; this one simply lands its own: for agentic work under real constraints, the small model on your device is not a fallback. Paired well with sparing cloud supervision, it’s frequently the better engine, and always the cheaper one.
